第五讲 - 集合与字典 (编程练习 1-2)

张建章

阿里巴巴商学院

杭州师范大学

2026-09



1 业务问题引入

2 集合：唯一成员

3 集合之间的关系

4 从集合到字典

5 字典：键值数据

6 字典的增删改查

7 综合业务实践

8 总结与拓展

Python 程序设计基础

第5讲 集合与字典 速记卡

从数据出发
发现更大的世界！

Better Data
Better Business
A Brighter You

用 Python
处理真实的商业数据
让世界更近！

Global Business

1 为什么学集合与字典？

实际的业务问题

比如两家店铺的海外市场：

```
store_a = ["泰国", "越南", "泰国", "印尼"]
store_b = ["越南", "印尼", "马来西亚"]
```

集合	字典
• 适合处理“有哪些不同成员”	• 适合处理“某个名称对应什么信息”

集合管去重和比较，字典管查询和记录。

2 集合：唯一成员

集合（set）是一组元素的集合：

- 元素唯一（不重复）
- 无序（顺序不重要）
- 可变（可以添加或删除元素）

自动去重
只保留唯一！

```
markets = ["泰国", "越南", "泰国", "印尼"]
unique_markets = set(markets)
# 结果: {'泰国', '印尼', '越南'}
```

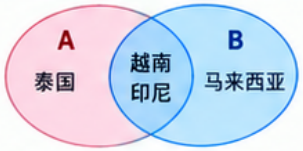
- 重复值会自动去掉
- 不能用索引和切片
- 空集合要写成 set()，{} 是空字典

常用操作：

```
"泰国" in unique_markets  # 成员判断
unique_markets.add("马来西亚")  # 添加
unique_markets.discard("泰国") # 删除(不存在也不报错)
len(unique_markets)  # 元素个数
```

3 集合之间的关系

```
store_a = {"泰国", "越南", "印尼"}
store_b = {"越南", "印尼", "马来西亚"}
```



并集	A | B	全部市场：泰国、越南、印尼、马来西亚
交集	A & B	共同市场：越南、印尼
差集	A - B	A独有：泰国
差集	B - A	B独有：马来西亚

差集有方向，前后顺序不同，结果会变。

4 从集合到字典

集合

关注“有哪些市场”

```
markets =
  {"泰国", "越南",
   "印尼"}
```

一组不重复的成员

字典

关注“市场对应什么信息”

```
market_sales = {
    "泰国": 128,
    "越南": 98,
    "印尼": 76,
}
```

每个市场有对应的数据

键 = 名称，值 = 对应的信息
字典通过“键-值对”来记录信息。

集合适合筛选成员，字典适合按名称查数据。

5 字典：键值数据

创建字典：

```
market_sales = {
    "泰国": 128,
    "越南": 98,
    "印尼": 76
}
```

业务记录示例：

```
order = {
    "order_id": "ORD-20261013-001",
    "market": "泰国",
    "product": "无线耳机",
    "amount": 299.90,
    "paid": True
}
```

字典就是
一张数据表！

- 每个键必须唯一
- 键说明字段含义
- 值可以是字符串、数值、布尔值等

6 字典的查、增、改、删

查：获取值

```
order["market"]        # '泰国'
order.get("amount")    # 299.90
order.get("coupon", "未使用优惠券")
```

增 / 改：添加或修改值

```
order["status"] = "PAID"   # 新增键值对
order["amount"] = 289.90   # 修改已有值
order.update({"currency": "CNY",
              "channel": "独立站"})  # 批量更新
```

删：删除键值对

```
removed_status = order.pop("status")
```

查看所有键、值、键值对

```
order.keys()     # 所有键
order.values()   # 所有值
order.items())   # 所有键值对
```

用 get()更稳妥；键可能不存在时，不容易报错。

7 综合业务实践

两家店铺的销售数据：

```
sales_a = {
    "泰国": 128,
    "越南": 98,
    "印尼": 76
}
```
店铺A

```
sales_b = {
    "越南": 105,
    "印尼": 69,
    "马来西亚": 63
}
```
店铺B

1. **取出键（获得市场列表）**
   `markets_a = set(sales_a.keys())`
2. **做集合比较**
   并集 / 交集 / 差集
3. **回到字典查数据**
   `sales_a["越南"]  # 98`
   `sales_b["越南"]  # 105`
4. **输出摘要（以“越南”为例）**

```
市场：越南
店铺A销售额：98万元
店铺B销售额：105万元
销售额差值（A-B）：-7万元
```

用数据
支持更好的
商业决策！

8 总结与易错点

核心总结

- 集合：去重、成员判断、群体比较
- 字典：字段记录、按键查询、更新数据

常见易错点

1. 空集合不是 {}，要写成 set()
2. 集合无序，不能按下标取值
3. 字典判断 in 时，检查的是“键”
4. 用 pop() 会删除键值对并返回被删的值

小扩展（了解一下）

- 对称差集 ^　　# 两边不同的成员
- issubset()　　# 是否为子集
- setdefault()　　# 如果键不存在则设置默认值

多练习，多用真实数据试一试！

数据连接世界
Python 创造更多可能！

记住：集合先帮你找“有哪些”，字典再帮你查“是什么”。
先分清数据结构，再写代码，商业数据处理会更轻松！

小小的代码
大大的世界！

海外市场数据中的两个问题

问题1：市场去重与比较

两家店铺记录了订单涉及的海外市场：

```
store_a = [
"泰国", "越南",
"泰国", "印尼"
]

store_b = [
"越南", "印尼",
"马来西亚"
]
```

- 店铺A覆盖了哪些不同的市场？
- 两家店铺共同覆盖哪些市场？
- 店铺B新增了哪些市场？

问题2：市场信息查询

每个市场还有对应的销售数据：

市场	销售额（万元）
泰国	128
越南	98
印尼	76
马来西亚	63

- 如何通过市场名称直接查询销售额？
- 如何增加新市场或修改已有数据？
- 如何把市场及其数据保存在一起？

本讲的学习主线：集合用于处理唯一成员及集合之间的关系；字典用于建立“键与值”的对应关系，并通过键查询业务信息。

集合：保留唯一成员

```
markets = [
"泰国", "越南",
"泰国", "印尼"
]

unique_markets = set(markets)

print(unique_markets)
# 元素的显示顺序可能不同

print(len(markets))          # 4
print(len(unique_markets))   # 3
```

为什么使用集合

列表中可以包含重复元素。集合中的每个元素只保留一次，因此适合处理市场名单、客户编号和商品标签等唯一成员。

集合的主要特点：元素唯一，重复值会被自动去除；集合不按照位置组织数据，不能使用索引和切片；集合本身可以修改，可以增加或删除元素。（唯一、无序、可变）

创建集合

```python
# 直接写出集合中的元素
markets = {"泰国", "越南", "印尼"}

# 把列表转换为集合，同时去除重复值
markets_from_list = set(["泰国", "越南", "泰国"])

# 创建空集合
empty_set = set()

print(type(markets))     # <class 'set'>
print(type(empty_set))   # <class 'set'>
print(type({}))          # <class 'dict'>
```

两种常用方式

已知具体元素时，可以使用花括号创建集合；需要从列表、元组或字符串转换时，使用 `set()`。空花括号 `{}` 创建的是空字典。空集合必须写成 `set()`。

集合元素的要求：集合元素必须可以保持不变。常用元素包括字符串、整数、浮点数和元组；列表与字典不能直接作为集合元素。

查询与更新集合

```
markets = {"泰国", "越南", "印尼"}

print("越南" in markets)          # True
print("马来西亚" in markets)      # False

markets.add("马来西亚")
markets.add("越南")               # 已存在，不会重复添加

markets.discard("泰国")
markets.discard("菲律宾")         # 不存在，也不会报错

print(markets)
print(len(markets))
```

常用操作

- `in` 判断某个成员是否存在，判断某个市场是否已经进入业务范围；
- `add()` 添加一个成员，已有成员不会重复出现；
- `discard()` 删除指定成员，成员不存在时不会报错；
- `len()` 返回集合中不同成员的数量。

并集与交集

```python
store_a = {"泰国", "越南", "印尼"}
store_b = {"越南", "印尼", "马来西亚"}

all_markets = store_a | store_b
shared_markets = store_a & store_b

print(all_markets)
# 显示顺序可能不同:
# {'泰国', '越南', '印尼', '马来西亚'}

print(shared_markets)
# {'越南', '印尼'}
```

并集：|

并集包含两个集合中的全部不同成员。本例中，并集表示两家店铺覆盖的所有海外市场。

交集：&

交集只包含两个集合共同拥有的成员。本例中，交集表示两家店铺共同覆盖的市场。

差集具有方向

```
store_a = {"泰国", "越南", "印尼"}
store_b = {"越南", "印尼", "马来西亚"}

only_a = store_a - store_b
only_b = store_b - store_a

print(only_a)  # {'泰国'}
print(only_b)  # {'马来西亚'}
```

差集：-

`store_a - store_b` 保留店铺A拥有、店铺B没有的成员。交换两个集合的位置，结果也会改变。

根据业务问题选择运算

- 查询两家店铺覆盖的全部市场：使用并集 `|`；
- 查询两家店铺共同覆盖的市场：使用交集 `&`；
- 查询某家店铺独有的市场：使用差集 `-`。

从成员集合到键值映射

集合：有哪些市场

```
markets =
↪  {"泰国","越南","印尼"}

print("越南" in markets) # True
```

集合保存唯一成员，可以判断某个市场是否存在，也可比较不同店铺的市场范围。

字典：市场对应什么信息

```
market_sales = {"泰国":
↪  128,"越南": 98,"印尼": 76}

print(market_sales["越南"]) # 98
```

字典把市场名称与销售额连接起来，可以通过市场名称直接查询对应数据。

键值对

字典中的一组对应关系称为“键值对”。在 `"越南": 98` 中，`"越南"` 是键，`98` 是值。

两种数据结构解决不同问题

集合关注唯一成员以及成员之间的关系；字典使用唯一的键组织和查询与该键相关的业务信息。

创建字典

```
market_sales = {
"泰国": 128,
"越南": 98,
"印尼": 76
}

empty_dict = {}

print(market_sales)
print(type(market_sales))  # <class 'dict'>
print(len(market_sales))   # 3
print(type(empty_dict))    # <class 'dict'>
```

字典的基本结构

字典使用花括号表示。键和值之间使用冒号分隔，多个键值对之间使用逗号分隔。

键和值的要求

每个键必须唯一，不能同时出现两个相同的键；键通常使用字符串、数值或元组等不可变数据；值可以使用字符串、数值、列表或其他字典；空花括号 `{}` 创建空字典，而不是空集合。

字典表示业务记录

```
order = {
"order_id": "ORD-20261013-001",
"market": "泰国",
"product": "无线耳机",
"price": 299.90,
"paid": True
}

print(order)
print(len(order))  # 5
```

字段名称作为键

`"order_id"`、`"market"`和`"price"`是字段名称，对应的订单数据是这些键的值。

字典的适用场景

当业务字段较多时，通过键读取数据通常更直观。一个字典可以包含不同类型的值，本例同时包含字符串、浮点数和布尔值，这些值共同描述同一条订单记录。

读取字典中的值

```
print(order["market"]) # 泰国

print(order.get("price")) # 299.9

print(order.get("coupon")) # None

print(order.get("coupon", "未使用优惠券")) # 未使用优惠券

print("price" in order) # True
```

使用方括号读取和使用get()读取

`order["market"]` 直接返回指定键对应的值。如果键不存在，程序会产生 `KeyError` 。键不存在时， `get()` 返回 `None` 或指定的默认值，并且不会修改原字典。

字典中的成员判断

表达式 `"price" in order` 检查的是键是否存在，不是检查值是否存在。

新增与修改键值对

```
order = {
"order_id": "ORD-20261013-001",
"market": "泰国",
"price": 299.90
}

# 键不存在：新增键值对
order["status"] = "PAID"

# 键已存在：修改对应的值
order["price"] = 289.90

# 一次增加或修改多个键值对
order.update({"currency": "CNY", "channel": "独立站"})

print(order)
```

如果键不存在，方括号赋值会新增键值对；如果键已经存在，方括号赋值会修改原来的值。`update()` 可以一次增加或修改多个键值对，传入的键已存在时，新值会替换旧值。

删除指定键值对

```
removed_status = order.pop("status")
print(removed_status) # PAID

coupon = order.pop(
"coupon",
"未使用优惠券"
)
print(coupon) # 未使用优惠券

print(order)
```

pop()的作用

`pop()` 删除指定键值对，并返回被删除的值。如果键不存在且没有提供默认值，将产生 `KeyError`；如果提供了默认值，则返回该默认值，不会产生 `KeyError`。

查看键、值和键值对

```
print(len(order))
print("amount" in order)

print(list(order.keys()))
print(list(order.values()))
print(list(order.items()))
```

三种查看方式

- `keys()` 返回全部键；
- `values()` 返回全部值；
- `items()` 返回全部键值对，每个键值对表现为一个元组。

这三个方法返回可以随字典变化的视图对象。本例使用 `list()` 转换后输出，便于观察其中的内容。

后续课程中的用途：学习循环后，可以依次处理 `keys()`、`values()` 或 `items()` 中的数据。

实践任务：两家店铺的海外市场

```
# 字典的键是海外市场，值是该市场的销售额，单位为万元。
sales_a = {
"泰国": 128,
"越南": 98,
"印尼": 76
}

sales_b = {
"越南": 105,
"印尼": 69,
"马来西亚": 63
}
```

实践任务

1. 找出两家店铺覆盖的全部市场；
2. 找出共同市场和各自独有的市场；
3. 查询越南市场的销售额；
4. 比较两家店铺在越南市场的销售表现。

数据结构的分工：字典保存“市场与销售额”的对应关系；集合处理市场之间的关系。

市场范围的集合分析

```
# keys()取得字典中的全部市场名称，
# set()把这些键转换为集合，以便执行集合运算。
markets_a = set(sales_a.keys())
markets_b = set(sales_b.keys())

all_markets = markets_a | markets_b
shared_markets = markets_a & markets_b
only_a = markets_a - markets_b
only_b = markets_b - markets_a

print(all_markets)
print(shared_markets)
print(only_a)
print(only_b)
```

分析结果

- 全部市场：泰国、越南、印尼和马来西亚；
- 共同市场：越南和印尼；
- 店铺A独有市场：泰国；
- 店铺B独有市场：马来西亚。

共同市场的销售额比较

```
focus_market = "越南"

print(focus_market in shared_markets) # True

sales_a_value = sales_a[focus_market]
sales_b_value = sales_b[focus_market]
sales_gap = sales_a_value - sales_b_value

summary = (
f"市场：{focus_market}\n"
f"店铺A销售额：{sales_a_value}万元\n"
f"店铺B销售额：{sales_b_value}万元\n"
f"销售额差值（A-B）：{sales_gap}万元"
)

print(summary)
```

完整处理过程

字典保存业务数据，集合筛选和比较市场，字典再根据市场名称返回销售额，最后生成摘要。

集合与字典的核心区别

比较内容	集合	字典
核心问题	有哪些不同成员，成员之间有什么关系	一个键对应什么业务信息
数据结构	每个元素唯一	每个键唯一，键对应一个值
常用操作	`set()`、`in`、`|`、`&`、`-`	方括号、`get()`、赋值、`pop()`
业务用途	去重、成员判断、群体比较	业务记录、信息查询、字段更新

数据类型的选择

只需要保存唯一成员或比较成员关系时，使用集合；需要建立“名称与信息”的对应关系时，使用字典。

集合与字典的协作

先用字典保存市场与销售额，再把字典键转换为集合进行比较，最后使用筛选出的键回到字典查询业务数据。

本讲的核心能力

根据业务问题选择数据结构，并把数据的保存、筛选、查询和输出组织为可以逐步检查的程序。

课后实践与可选拓展

```
available_skus = {"SKU-A01", "SKU-B02", "SKU-C03"}

promotion_skus = {"SKU-B02", "SKU-C03", "SKU-D04"}

prices = {
"SKU-A01": 129.90, "SKU-B02": 89.00,
"SKU-C03": 199.00, "SKU-D04": 59.00
}
```

课后实践

1. 找出同时有库存并参加促销的商品；
2. 查询这些商品的价格并计算价格总和；
3. 把 `SKU-D04` 加入库存集合；
4. 把 `SKU-B02` 的价格修改为 `85.00` 元；
5. 使用 `f-string` 输出处理结果。

修改数据前，符合条件的商品是 `SKU-B02` 和 `SKU-C03`，价格总和为 `288.00` 元。

课后拓展

- 对称差集 `^`：找出只出现在一个集合中的成员；
- `issubset()`：检查一个集合是否完全包含在另一个集合中；
- `setdefault()`：键不存在时插入默认值，并返回该值；
- 字典复制：比较赋值、浅复制和深复制。

THE END